

Microservices with Spring Boot 3.0

1. User and Order Management System

User Service

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

application.properties

```
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/userdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

User.java

```
package com.cognizant.userservice.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
public class User {
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
}

```

UserRepository.java

```

package com.cognizant.userservice.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.cognizant.userservice.entity.User;

public interface UserRepository extends JpaRepository<User, Long> {
}

```

UserController.java

```

package com.cognizant.userservice.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.cognizant.userservice.entity.User;
import com.cognizant.userservice.repository.UserRepository;

@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserRepository userRepository;

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User user) {
        return new ResponseEntity<>(userRepository.save(user), HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    public ResponseEntity<User> getById(@PathVariable Long id) {
        return userRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}

```

Order Service

pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>

```

application.properties

```

server.port=8082
spring.datasource.url=jdbc:mysql://localhost:3306/orderdb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
user-service.url=http://localhost:8081

```

Order.java

```

package com.cognizant.orderservice.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

```

        private Long userId;
        private String product;
        private Integer quantity;
        private String status;
    }

```

OrderRepository.java

```

package com.cognizant.orderservice.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.cognizant.orderservice.entity.Order;

public interface OrderRepository extends JpaRepository<Order, Long> {
}

```

UserNotFoundException.java

```

package com.cognizant.orderservice.exception;

public class UserNotFoundException extends RuntimeException {

    public UserNotFoundException(String message) {
        super(message);
    }
}

```

WebClientConfig.java

```

package com.cognizant.orderservice.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.client.WebClient;

@Configuration
public class WebClientConfig {

    @Bean
    public WebClient.Builder webClientBuilder() {
        return WebClient.builder();
    }
}

```

OrderService.java

```

package com.cognizant.orderservice.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;
import org.springframework.web.reactive.function.client.WebClientResponseException;

```

```

import com.cognizant.orderservice.entity.Order;
import com.cognizant.orderservice.exception.UserNotFoundException;
import com.cognizant.orderservice.repository.OrderRepository;

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private WebClient.Builder webClientBuilder;

    @Value("${user-service.url}")
    private String userServiceUrl;

    public Order createOrder(Order order) {
        try {
            webClientBuilder.build()
                .get()
                .uri(userServiceUrl + "/users/{id}", order.getUserId())
                .retrieve()
                .toBodilessEntity()
                .block();
        } catch (WebClientResponseException.NotFound ex) {
            throw new UserNotFoundException("User with id " + order.getUserId() + " not
found");
        }
        order.setStatus("CONFIRMED");
        return orderRepository.save(order);
    }
}

```

GlobalExceptionHandler.java

```

package com.cognizant.orderservice.exception;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleUserNotFound(UserNotFoundException
ex) {
        Map<String, Object> body = new HashMap<>();
    }
}

```

```

        body.put("timestamp", Instant.now().toString());
        body.put("status", 404);
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());
        body.put("path", "/orders");
        return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
    }
}

```

OrderController.java

```

package com.cognizant.orderservice.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.cognizant.orderservice.entity.Order;
import com.cognizant.orderservice.service.OrderService;

@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    private OrderService orderService;

    @PostMapping
    public ResponseEntity<Order> create(@RequestBody Order order) {
        return new ResponseEntity<>(orderService.createOrder(order),
        HttpStatus.CREATED);
    }
}

```

Request – create a user

```

curl -s -X POST http://localhost:8081/users -H "Content-Type: application/json" \
-d '{"name":"Priya Sharma","email":"priya.sharma@example.com"}'

```

Output

```

{"id":1,"name":"Priya Sharma","email":"priya.sharma@example.com"}

```

Request – place an order for that user

```

curl -s -X POST http://localhost:8082/orders -H "Content-Type: application/json" \
-d '{"userId":1,"product":"Wireless Mouse","quantity":2}'

```

Output

```

{"id":1,"userId":1,"product":"Wireless Mouse","quantity":2,"status":"CONFIRMED"}

```

Request – place an order for a non-existent user

```

curl -s -X POST http://localhost:8082/orders -H "Content-Type: application/json" \
-d '{"userId":99,"product":"Keyboard","quantity":1}'

```

Output

```
{"timestamp":"2026-07-23T06:40:12.501Z","status":404,"error":"Not Found","message":"User with id 99 not found","path":"/orders"}
```

2. Inventory Management System with Service Discovery

Eureka Server

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

EurekaServerApplication.java

```
package com.cognizant.eurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml

```
server:
  port: 8761
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

Config Server

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

ConfigServerApplication.java

```
package com.cognizant.configserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;
```

```

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

```

application.yml

```

server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/cognizant-training/microservices-config

```

Product Service

pom.xml

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

```

application.yml

```

server:
  port: 8083
spring:

```



```

application:
  name: product-service
config:
  import: configserver:http://localhost:8888
datasource:
  url: jdbc:mysql://localhost:3306/productdb
  username: root
  password: root
jpa:
  hibernate:
    ddl-auto: update
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka

```

Product.java

```

package com.cognizant.productservice.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double price;
}

```

ProductRepository.java

```

package com.cognizant.productservice.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.cognizant.productservice.entity.Product;

public interface ProductRepository extends JpaRepository<Product, Long> {
}

```

ProductController.java

```

package com.cognizant.productservice.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.cognizant.productservice.entity.Product;
import com.cognizant.productservice.repository.ProductRepository;

@RestController
@RequestMapping("/products")
public class ProductController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping("/{id}")
    public Product getById(@PathVariable Long id) {
        return productRepository.findById(id).orElseThrow();
    }
}

```

Inventory Service

pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>

```

```
    <optional>true</optional>
</dependency>
```

application.yml

```
server:
  port: 8084
spring:
  application:
    name: inventory-service
  config:
    import: configserver:http://localhost:8888
  datasource:
    url: jdbc:mysql://localhost:3306/inventorydb
    username: root
    password: root
  jpa:
    hibernate:
      ddl-auto: update
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Inventory.java

```
package com.cognizant.inventoryservice.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@NoArgsConstructor
public class Inventory {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long productId;
    private Integer stockQuantity;
}
```

InventoryRepository.java

```
package com.cognizant.inventoryservice.repository;

import java.util.Optional;
```

```
import org.springframework.data.jpa.repository.JpaRepository;

import com.cognizant.inventoryservice.entity.Inventory;

public interface InventoryRepository extends JpaRepository<Inventory, Long> {

    Optional<Inventory> findByProductId(Long productId);
}
```

ProductDTO.java

```
package com.cognizant.inventoryservice.dto;

import lombok.Data;

@Data
public class ProductDTO {

    private Long id;
    private String name;
    private Double price;
}
```

ProductClient.java

```
package com.cognizant.inventoryservice.client;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

import com.cognizant.inventoryservice.dto.ProductDTO;

@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping("/products/{id}")
    ProductDTO getProduct(@PathVariable Long id);
}
```

InventoryController.java

```
package com.cognizant.inventoryservice.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import com.cognizant.inventoryservice.client.ProductClient;
import com.cognizant.inventoryservice.dto.ProductDTO;
import com.cognizant.inventoryservice.repository.InventoryRepository;
```

```

@RestController
@RequestMapping("/inventory")
public class InventoryController {

    @Autowired
    private InventoryRepository inventoryRepository;

    @Autowired
    private ProductClient productClient;

    @GetMapping("/{productId}")
    public Map<String, Object> getStock(@PathVariable Long productId) {
        ProductDTO product = productClient.getProduct(productId);
        int stock = inventoryRepository.findByProductId(productId)
            .map(inv -> inv.getStockQuantity())
            .orElse(0);

        Map<String, Object> response = new HashMap<>();
        response.put("productId", product.getId());
        response.put("productName", product.getName());
        response.put("price", product.getPrice());
        response.put("stockQuantity", stock);
        return response;
    }
}

```

InventoryServiceApplication.java

```

package com.cognizant.inventoryservice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;

@SpringBootApplication
@EnableFeignClients
public class InventoryServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(InventoryServiceApplication.class, args);
    }
}

```

Request – Eureka registered applications

```
curl -s http://localhost:8761/eureka/apps
```

Output

```

PRODUCT-SERVICE      (1) - localhost:product-service:8083    UP
INVENTORY-SERVICE    (1) - localhost:inventory-service:8084  UP

```

Request – get stock for a product

```
curl -s http://localhost:8084/inventory/1
```

Output

```
{"productId":1,"productName":"Wireless Mouse","price":599.0,"stockQuantity":120}
```

3. API Gateway

pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis-reactive</artifactId>
</dependency>
```

application.yml

```
server:
  port: 8080
spring:
  data:
    redis:
      host: localhost
      port: 6379
  cloud:
    gateway:
      filter:
        local-response-cache:
          enabled: true
          time-to-live: 30s
      routes:
        - id: customer-service
          uri: http://localhost:8085
          predicates:
            - Path=/customers/**
          filters:
            - RewritePath=/customers/(?<segment>.*), /api/customers/${segment}
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
            - LocalResponseCache
        - id: billing-service
          uri: http://localhost:8086
          predicates:
            - Path=/billing/**
          filters:
            - RewritePath=/billing/(?<segment>.*), /api/billing/${segment}
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
            - LocalResponseCache
```

RateLimiterConfig.java

```
package com.cognizant.apigateway.config;

import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import reactor.core.publisher.Mono;

@Configuration
public class RateLimiterConfig {

    @Bean
    public KeyResolver clientKeyResolver() {
        return exchange -> Mono.just(
            exchange.getRequest().getRemoteAddress().getAddress().getHostAddress());
    }
}
```

Request – gateway routes and rewrites the path

```
curl -s -v http://localhost:8080/customers/101
```

Output

```
< HTTP/1.1 200 OK
< Content-Type: application/json
{"id":101,"name":"Anita Verma"}
```

The Path=/customers/** predicate matches the request, RewritePath forwards it to http://localhost:8085/api/customers/101, and the response is cached by LocalResponseCache for 30 seconds.

Request – 12 rapid requests to trigger the rate limiter

```
for i in $(seq 1 12); do curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:8080/customers/101; done
```

Output

```
200
200
200
200
200
200
200
200
200
200
429
429
```

4. Resilient Microservices with Circuit Breaker

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

application.yml

```
server:
  port: 8087
resilience4j:
  circuitbreaker:
    instances:
      thirdPartyApi:
        sliding-window-size: 10
        failure-rate-threshold: 50
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 3
        register-health-indicator: true
  timelimiter:
    instances:
      thirdPartyApi:
        timeout-duration: 2s
```

PaymentRequest.java

```
package com.cognizant.paymentservice.dto;

import lombok.Data;

@Data
public class PaymentRequest {

    private Long orderId;
    private Double amount;
}
```

PaymentService.java

```
package com.cognizant.paymentservice.service;

import java.time.Duration;
import java.util.HashMap;
import java.util.Map;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```



```

import org.springframework.stereotype.Service;
import org.springframework.web.reactive.function.client.WebClient;

import com.cognizant.paymentservice.dto.PaymentRequest;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;

@Service
public class PaymentService {

    private static final Logger LOGGER = LoggerFactory.getLogger(PaymentService.class);

    private final WebClient webClient = WebClient.create("https://slow-third-party-api.example.com");

    @CircuitBreaker(name = "thirdPartyApi", fallbackMethod = "paymentFallback")
    public Map<String, Object> processPayment(PaymentRequest request) {
        Map<String, Object> gatewayResponse = webClient.post()
            .uri("/charge")
            .bodyValue(request)
            .retrieve()
            .bodyToMono(Map.class)
            .block(Duration.ofSeconds(2));

        Map<String, Object> response = new HashMap<>();
        response.put("orderId", request.getOrderId());
        response.put("status", "SUCCESS");
        response.put("transactionId", gatewayResponse.get("transactionId"));
        return response;
    }

    public Map<String, Object> paymentFallback(PaymentRequest request, Throwable throwable) {
        LOGGER.warn("Fallback triggered for orderId {}: {}", request.getOrderId(),
            throwable.toString());

        Map<String, Object> response = new HashMap<>();
        response.put("orderId", request.getOrderId());
        response.put("status", "FAILED");
        response.put("message", "Payment gateway is currently unavailable, please try again later");
        return response;
    }
}

```

PaymentController.java

```

package com.cognizant.paymentservice.controller;

import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

```

```

import com.cognizant.paymentservice.dto.PaymentRequest;
import com.cognizant.paymentservice.service.PaymentService;

@RestController
@RequestMapping("/payments")
public class PaymentController {

    @Autowired
    private PaymentService paymentService;

    @PostMapping("/process")
    public Map<String, Object> process(@RequestBody PaymentRequest request) {
        return paymentService.processPayment(request);
    }
}

```

Request – third-party API responding normally

```

curl -s -X POST http://localhost:8087/payments/process -H "Content-Type:
application/json" \
-d '{"orderId":501,"amount":2500}'

```

Output

```

{"orderId":501,"status":"SUCCESS","transactionId":"TXN10023"}

```

Request – repeated after the third-party API starts timing out

```

curl -s -X POST http://localhost:8087/payments/process -H "Content-Type:
application/json" \
-d '{"orderId":501,"amount":2500}'

```

Output

```

{"orderId":501,"status":"FAILED","message":"Payment gateway is currently unavailable,
please try again later"}

```

Log output

```

WARN PaymentService - Fallback triggered for orderId 501:
io.github.resilience4j.circuitbreaker.CallNotPermittedException: CircuitBreaker
'thirdPartyApi' is OPEN

```